

Аспекты регионализации

Аспекты регионализации — это константы, значения которых зависят не от языка приложения, а от региона, в котором это приложение развернуто. К ним относятся:

- [ИМЯ ЭКОСИСТЕМЫ](#);
- [идентификационный номер](#) (ИНН/БИН);
- [значок валюты](#);
- [название валюты](#).

Аспекты регионализации используются или в виде плейсхолдеров для переводов, или в виде функций.

Использование функций

Фреймворк Saby предоставляет набор функций, которые позволят получать значения аспектов во время работы приложения (в run-time).

Использование плейсхолдеров

Плейсхолдер — это специальный набор символов, используемый внутри перевода, который ядро локализации заменит на значение соответствующего аспекта во время выполнения программы. Для каждого аспекта существует свой набор подстановок.

Например, вызов

```
1 # Python
2 rk( "Отзывы о %notranslates_ecos_name%" )

1 // C++
2 rk( L"Отзывы о %notranslates_ecos_name%" )
```

вернет "Отзывы о Saby" или "Отзывы о Setty" в зависимости от региона, в котором развернуто приложение.

Название аспекта	Подстановка	Значение для России	Значение для Казахстана
ИМЯ ЭКОСИСТЕМЫ	%notranslates_ecos_name%	Saby	Setty
значок валюты	%notranslates_cur_sign%	₽	₸
идентификационный номер	%notranslates_inn_name%	ИНН	БИН
название валюты	%notranslates_cur_name_one %	рубль	тенге
	%notranslates_cur_name_one _cap%	Рубль	Тенге
	%notranslates_cur_name_plural% /	рубли	тенге
	%notranslates_cur_name_plural_cap%	Рубли	Тенге

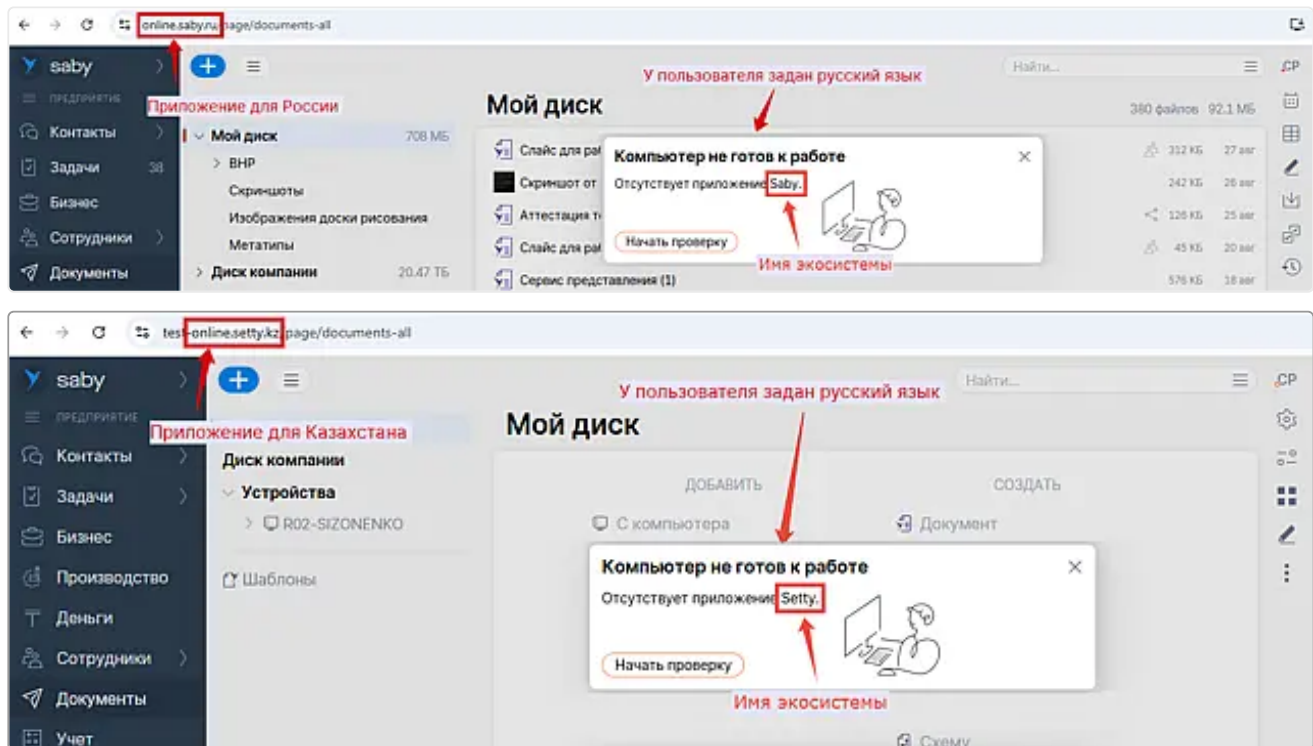
Замена плейсхолдеров происходит автоматически. Со стороны бизнес-логики на сервере используется run-time подстановка во время вызова rk(). В оффлайн-приложениях используется compile-time подстановка значений во время компиляции ресурсов.

На UI используется compile-time подстановка значений во время обработки словарей saby-builder'ом.

[Подробнее о замене плейсхолдеров](#)

Имя экосистемы

В различных регионах имя экосистемы отличается, например, для России и Узбекистана - Saby, для Казахстана - Setty.



Независимо от того, в какой экосистеме работает пользователь, у него задан русский язык.

Плейсхолдер

```
1 # Python
2 import sbis
3 def GetMessage():
4     return sbis.rk("Отсутствует приложение %notranslates_ecos_name%.")

1 // C++
2 String GetMessage()
3     return sbis::rk( L"Отсутствует приложение %notranslates_ecos_name%."
4 );

1 // TypeScript
2 import * as rk from 'i18n!UIModuleName';
3 return rk('Отсутствует приложение %notranslates_ecos_name%.');
```

Для автоматической подстановки названия экосистемы в пользовательское сообщение добавьте в словарь переводов ключ "Отсутствует приложение %notranslates_ecos_name%" с использованием плейсхолдера %notranslates_ecos_name%.

Пример для русского, казахского и английского словарей:

```
1 // ru.json
2 "Отсутствует приложение %notranslates_ecos_name%." : "Отсутствует
приложение %notranslates_ecos_name%."
3
4 // kk.json
5 "Отсутствует приложение %notranslates_ecos_name%." :
"%notranslates_ecos_name% қолданбасы жоқ."
6
7 // en.json
8 "Отсутствует приложение %notranslates_ecos_name%." :
```

```
"%nottranslates_ecos_name% app is missing."
```

Функция

Для получения имени экосистемы в платформе предусмотрены функции:

- для бизнес-логики: `CurrentEcosystemName()`

- [C++](#)

```
1 {
2     StringView const ecosystem_name = sbis::CurrentEcosystemName();
3 }
```

- [Python](#)

```
ecosystem_name = sbis.CurrentEcosystemName();
```

- для UI: [currentLocaleConfig](#)

```
1 import {controller} from 'I18n/i18n';
2 const ecosystemName = controller.currentLocaleConfig.ecosystem.name;
```

Идентификационный номер

Для идентификации юридических лиц в России используется ИНН (индивидуальный номер налогоплательщика), в Казахстане - БИН (бизнес-идентификационный номер):

Плейсхолдер

```
1 # Python
2 import sbis
3 def GetMessage(inn_value):
4     return sbis.rk("Ваш %nottranslates_inn_name% {}").format(inn_value)
5
1 // C++
2 String GetMessage( StringView inn_value )
3 {
4     String const translated = sbis::rk( L"Ваш %nottranslates_inn_name%
    %1%" );
5     return sbis::Format( StringView( translated ), inn_value );
6 }
7
1 // TypeScript
2 import * as rk from 'i18n!UIModuleName';
3 const getMessage = (innValue: string): string => {
4     const translated = sbis.rk("Ваш %nottranslates_inn_name% %1%");
5     return sbis.format(translated, innValue);
6 };
```

Для автоматической подстановки ИНН добавьте в словарь перевод ключа "Ваш %nottranslates_ecos_name% {}" с использованием плейсхолдера `%nottranslates_inn_name%`.

```
1 // kk.json
2 "Ваш %nottranslates_inn_name% {}": "Сіздің %nottranslates_inn_name% {}"
3
4 // en.json
5 "Ваш %nottranslates_inn_name% {}": "Yours %nottranslates_inn_name% {}"
```



В C++ и Python разные способы указания плейсхолдеров, в примере указаны ключи для Python. Для C++ {} меняется на %1% и ключ будет: "Ваш %nottranslates_inn_name% %1%"

Функция

Для получения идентификационного номера в платформе предусмотрены функции:

- для бизнес-логики: [CurrentTINName\(\)](#)

- [C++](#)

```
1 {
2     String const tinn_name = sbis::CurrentTINName();
3 }
```

- [Python](#)

```
1 tin_name = sbis.CurrentTINName();
```

- для UI: [currentLocaleConfig](#)

```
1 import {controller} from 'I18n/i18n';
2 const innName = controller.currentLocaleConfig.ecosystem.TINName;
```

Значок валюты

Значок региональной валюты меняется в зависимости от страны, например, для России - Р (рубль), а для Казахстана - Т (тенге):

Плейсхолдер

```
1 # Python
2 import sbis
3 def GetMessage():
4     return sbis.rk("Аванс 100000 %notranslates_cur_sign%")

1 // C++
2 String GetMessage()
3     return sbis::rk( L"Аванс 100000 %notranslates_cur_sign%" );

1 // TypeScript
2 import * as rk from 'i18n!UIModuleName';
3 return rk('Аванс 100000 %notranslates_cur_sign%');
```

Для автоматической подстановки значка валюты добавьте в словарь перевод ключа "Аванс 100000 %notranslates_cur_sign%".

Пример для русского, казахского и английского словарей:

```
1 // ru.json
2 "Оплата в %notranslates_cur_sign%": "Оплата в %notranslates_cur_sign%"
3
4 // kk.json
5 "Оплата в %notranslates_cur_sign%": "Төлем %notranslates_cur_sign%"
6
7 // en.json
8 "Оплата в %notranslates_cur_sign%": "Payment in %notranslates_cur_sign%"
```

Функция

Для получения значка валюты в платформе предусмотрены функции:

- для бизнес-логики: [CurrentCurrencySign\(\)](#)

- [C++](#)

```
1 | Format( rk( L"Оплата в %1%"_sv ), CurrentCurrencySign() ); //
```

Оплата в ₸

- Python

```
1 | result = f"Оплата в {CurrentCurrencySign()}" # Оплата в ₸
```

- для UI: [currentLocaleConfig](#)

```
1 | import {controller} from 'I18n/i18n';
2 | const currencySymbol = controller.currentLocaleConfig.money.symbol;
```

Если у валюты нет значка, метод возвращает пустую строку.

Название валюты

Название валюты используемой в каждом конкретном регионе может отличаться: в России - рубль, в Казахстане - тенге.

Плейсхолдеры

Атоматическая подстановка названия валюты поддерживается только для двух случаев:

- в единственном числе именительного падежа с заглавной/строчной буквы;
- во множественном числе именительного падежа с заглавной/строчной буквы.

```
1 | # Python
2 | import sbis
3 | def GetMessage():
4 |     return sbis.rk("Текущая валюта региона %notranslates_cur_name_one%")

1 | // C++
2 | String GetMessage()
3 |     return sbis::rk( "Текущая валюта региона
   | %notranslates_cur_name_one%" );

1 | // TypeScript
2 | import * as rk from 'i18n!UIModuleName';
3 | return rk("Текущая валюта региона %notranslates_cur_name_one%");
```

Для автоматической подстановки названия валюты добавьте в словарь перевод ключа "451 рубль" с использованием плейсхолдеров:

- %notranslates_cur_name_one%** - для перевода со строчной буквой
- %notranslates_cur_name_one_cap%** - для перевода с заглавной буквой

Пример для русского, казахского и английского словарей:

```
1 | // ru.json
2 | "Валюта региона %notranslates_cur_name_one%": "Валюта региона
   | %notranslates_cur_name_one%"
3 |
4 | // kk.json
5 | "Валюта региона %notranslates_cur_name_one%": "Аймақ валютасы
   | %notranslates_cur_name_one%"
6 |
7 | // en.json
8 | "Валюта региона %notranslates_cur_name_one%": "Currency of the region
   | %notranslates_cur_name_one%"
```

Для множественного числа используются плейсхолдеры **%notranslates_cur_name_plural%** (со строчной буквой) и **%notranslates_cur_name_plural_cap%** (с заглавной буквой):

```

1 // ru.json
2 "за публи": "за %nottranslates_cur_name_plural%"
3
4 // kk.json
5 "за публи": "за %nottranslates_cur_name_plural%"
6
7 // en.json
8 "за публи": "за %nottranslates_cur_name_plural%"

```

Функции

Для получения названия валюты в платформе предусмотрены функции:

- для бизнес-логики: `Loc( Decimal, CurrencyTranslationMode, CurrencyTranslationSize )`, где `CurrencyTranslationMode` - enum-режим вывода разменных денежных единиц отдельно или в составе номинала.

```

1 SPLIT, // запись в виде 100 рублей 25 копеек
2 NO_SPLIT, // запись в виде 100.25 рублей

```

`CurrencyTranslationSize` - enum-используемое сокращение для валюты.

```

1 SIGN_ONLY, // 100 ₸
2 MINIMAL, // 100 р. || 100 р. 25 к.
3 SHORT, // 100 руб. || 100 руб. 25 коп.
4 FULL, // 100 рублей || 100 рублей 25 копеек

```

Пример:

◦ [C++](#)

```

1 #include <sbis-lib/types/currency_translation.hpp>
2 #include <sbis-lib/utils/stringify/loc/decimal.hpp>
3
4 // ...
5
6 Decimal value( 100.25f );
7 Format( rk( L"У вас: %1%"_sv ), Loc( value,
8   CurrencyTranslationMode::SPLIT, CurrencyTranslationSize::FULL ) );
9
10 // "У вас: 100 рублей 25 копеек" для ru-RU
11 // "Сізде: 100 теңге 25 тиын" для kk-KZ

```

◦ [Python](#)

```

1 from decimal import Decimal
2
3 # ...
4
5 value = Decimal('100.25')
6 formatted_message = Format(
7     rk("У вас: %1%"),
8     Loc(value, CurrencyTranslationMode.SPLIT,
9       CurrencyTranslationSize.FULL)
10 )
11
12 # "У вас: 100 рублей 25 копеек" для ru-RU
13 # "Сізде: 100 теңге 25 тиын" для kk-KZ

```

- для UI: [Types/formatter:money](#)

```

1 import { money } from 'Types/formatter';
2
3 // 1000 рублей 55 копеек
4 money(1000.55, {

```

```

5     currencyName: CurrencyName.Full,
6     subunit: MoneyDisplayMode.Literal
7 });
8
9 // 1 рубль 00 копеек
10 money(1, {
11     currencyName: CurrencyName.Full,
12     subunit: MoneyDisplayMode.Literal
13 });

```

Для форматирования денежных данных в мобильных приложениях используйте методы:

- для iOS: [CurrencyFormatter.getMoney\(from value: NSNumber, mode: CurrencyTranslationMode, size: CurrencyTranslationSize\)](#), где [CurrencyTranslationMode](#) - enum-режим вывода разменных денежных единиц отдельно или в составе номинала.

```

1 split, // запись в виде 100 рублей 25 копеек
2 separated, // запись в виде 100.25 рублей

```

[CurrencyTranslationSize](#) - enum-используемое сокращение для валюты.

```

1 sign, // 100 ₸ || 100₸ 25к. || 100.25₸
2 min, // 100 p. || 100 p. 25 к. || 100.25p.
3 medium, // 100 руб. || 100 руб. 25 коп. || 100.25 руб.
4 full, // 100 рублей || 100 рублей 25 копеек || 100.25 рублей

```

Пример:

```

1 CurrencyFormatter.getMoney(from: 100.25, mode: .split, size: .full)
2
3 // en-US
4 // "100 dollars 67 cents"
5
6 // ru-KZ
7 // "100 тенге 67 тиын"
8
9 // en-UZ
10 // "100 soums 67 tiyins"

```

- для Android: [CurrencyFormatter.getMoney\(value: Number, mode: CurrencyTranslationMode, size: CurrencyTranslationSize\)](#), где [CurrencyTranslationMode](#) - enum-режим вывода разменных денежных единиц отдельно или в составе номинала.

```

1 // Разделенный пробелом (100 рублей 25 копеек).
2 SPLIT,
3 // Разделенный сепаратором (точка/запятая) (100.25 рублей).
4 SEPARATED

```

[CurrencyTranslationSize](#) - enum-используемое сокращение для валюты.

```

1 // Значок валюты (100₸).
2 SIGN,
3 // Минимальный (100 p.; 100 p. 25 к.).
4 MIN,
5 // Средний (100 руб.; 100 руб. 25 коп.).
6 MEDIUM,
7 // Полный (100 рублей; 100 рублей 25 копеек).
8 FULL

```

Пример:

```

1 CurrencyFormatter.getMoney(100.25, CurrencyTranslationMode.SPLIT,
2 CurrencyTranslationSize.FULL)
3
4 // en-US

```

```
4 // "100 dollars 25 cents"
5
6 // ru-KZ
7 // "100 теңге 25 тиын"
8
9 // en-UZ
10 // "100 soums 25 tiyins"
```